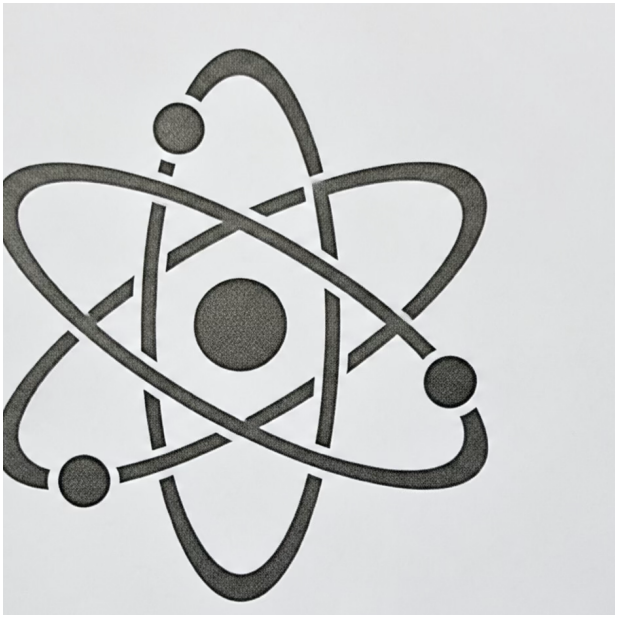
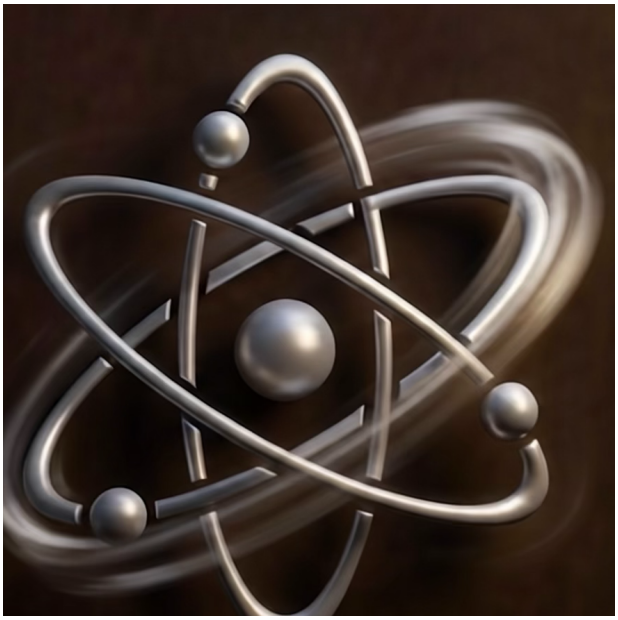
Hyperbolic Multi-Model Manifolds: Atoms, Toroidal Phase, and the Hyperbolic Nesting of the KeyValue-Cache



Left: flat mark, print.



Right: three-dimensional mark, display.

Atoma Labs

A map you can walk, a cache that knows where it lives, and an atom that earns its place.

The mind is a curved map. Knowledge sits in basins on it. The key-value cache sits inside it, not beside it. Nothing enters unjudged.

Stewart Barteau

AtomaLabs | Ares - SpecOps

SCIENTIFIC PREPRINT

November 4, 2026

https://atoma-labs-site.pages.dev

Front: plain language. Back: proofs. Numbers in parentheses, for example (3), refer to proofs in the Appendix, not to outside papers. Theory is labelled THEORY until it is measured.

Front: in plain language

1 The mind is a map, not a pile

Most systems store knowledge as a pile: a list, a table, a heap of vectors. A pile has no shape, so it cannot tell you what is near, what is far, or what belongs together. This paper starts somewhere else. The mind is a map, and the map is curved (1).

The map is the Lorentz hyperboloid: a surface that bends away from flatness in a precise way, so that distance means something. On that map, knowledge gathers in basins. A basin is a region around a centre, like a valley that holds what belongs to it. In this theory, those basins are Whitehead spheres: closed regions on the map that hold a family of related atoms together (1). If you know where a basin is, you know what it is near and what it is far from, without searching the whole pile.

That is the whole bet, stated plainly: shape carries meaning. If the shape is wrong, the map is wrong. If the shape is right, retrieval stops being a hunt and becomes a walk.

2 Atoms: one in, judged, or nothing in

Nothing enters the map on a claim. It enters as an atom, and an atom is the smallest unit the system will store (3).

The atom-intake gate is strict, and it is strict in a way you can check. A run that passes admits exactly one atom. Not roughly one. Exactly one. The gate is deterministic, which means the same inputs give the same answer every time. And it is judge-gated, which means a judge that is not the proposer decides whether the atom passes.

This gate is not the prompt-adoption gate. That is a different gate, with a different job: whether a prompt, a rule, or a piece of guidance is adopted for future use. Intake decides what knowledge enters. Adoption decides what guidance is followed. Mixing them up is how a system ends up believing its own instructions, or following its own memories as if they were orders. Keep them separate, by name and by job (3).

A system that lets anything in will fill up with noise and then call the noise knowledge. A system that admits one judged atom at a time stays clean, slowly, and can say why every atom is there.

3 Toroidal phase: a coherence score, nothing more

Atoms do not just sit in basins. They turn. Each atom carries phase, like a hand on a dial, and atoms that turn together belong together. Toroidal phase is the paper's way of saying that: selection is weighted by how well phases line up (2).

The score is simple, and the paper keeps it simple on purpose:

$$s = \cos(\Delta\theta_f) + \cos(\Delta\theta_s). \tag{1}$$

Two phase differences, fast and slow, each turned into a cosine, added together. When both phases line up, the score is at its highest, 2. When both are opposite, it is at its lowest, -2. Atoms with high scores are weighted up in selection. Atoms with low scores are weighted down.

That is all it is. It is a coherence score. It is not an attention layer. It does not replace the transformer's attention, it does not replace rotary position embeddings, and it has no end-to-end benchmark proving that it could. Anyone who tells you otherwise is upgrading a score into a claim it has never earned (2). The score is useful if it picks better atoms. Whether it does, at scale, is a measurement that has not been run yet, and the paper says so rather than pretending otherwise.

4 The cache lives inside the map

A language model remembers what it has already computed in its key-value cache, the KV cache. In most systems, that cache sits beside the model, as a separate buffer that fills up and gets emptied. In this theory, the cache sits inside the map itself (4).

That one move changes everything about how the cache behaves. KV pages are not just stored. They are placed. Each page sits at a position on the map, and its position is its future. The rule is: radius equals predicted reuse. A page the system expects to reuse a lot sits near the centre. A page it expects to reuse rarely sits near the edge. Reuse is not a guess made once; it is a prediction the system keeps updating as the run goes on (4).

So the cache is no longer a buffer you manage by hand. It is a landscape you read. Look at where a page sits, and you know how much the system thinks it will matter again.

5 Geometry does the evicting

Every cache fills up. The question is what leaves, and how. The usual answer is deletion: pick a page, throw it away, compact what remains. That is surgery on memory, and it is how a context window gets lobotomised, a piece at a time, until it can no longer remember why it was thinking what it was thinking.

This theory gives a different answer: eviction by geometry, not deletion (4). Low-reuse pages drift toward the boundary of the map. When they reach the boundary, they fall off. No one deletes them. No one compacts the rest. They leave because their position says they no longer belong.

And the reverse is just as important. A page that is needed again is refilled closest to the centre, where reuse is highest. A page that is being reused needs no eviction and no compaction at all. It stays, because it is being used, and use is the only justification a cache page ever needed. That is the anti-lobotomy of the context window: memory is lost by drifting away, not by being cut out, and what is kept is kept because it is alive in the run (4).

6 Three stores, three jobs

A map needs a home, and a home needs discipline. This system uses three stores, each for one job, and never collapses them into one (5).

SurrealDB holds the tensor KV nested in the multi-model map. That is the map itself, with its cache inside it. Qdrant holds retrieval: memories and facts with provenance, searched by regular BM25 plus cosine similarity, exactly as it already is. HyperDX holds telemetry: what happened, when, and in what order, so a run can be observed and audited.

Three stores by job, never collapsed. Collapsing them would be convenient, and it would be wrong: a map

is not a search index, a search index is not a log, and a log is not a map. Each store is judged by whether it does its own job, not by whether it could pretend to do another's (5).

The same discipline bounds the Glimmer result. The Glimmer key-value gate that passed is a prefix witness: it proves that a prefix can be recalled, bit for bit, through the gate. It does not prove distilled knowledge, it does not prove answer quality, and it does not prove that the same gate works for any model. A witness is a witness. It proves what it saw, and nothing more (5).

7 What is not claimed

A paper that only lists what it claims is half a paper. Here is the other half, in full.

This paper does not claim proven cross-model transfer: that an atom, a basin, or a KV placement learned on one model works on another. It does not claim a proven answer-quality gain from hyperbolic nesting, toroidal phase, or any other part of this theory. It does not claim an end-to-end toroidal attention replacement, because no such benchmark exists. It does not claim that retrieval was converted to hyperbolic form: regular BM25 plus cosine retrieval stays as it is, by the ruling of 2026-10-27, and any future conversion would be a new claim needing its own proof. It does not claim that mixed-curvature radius-versus-reuse tracking, batch or memory at equal quality, or boundary numerics near radius 1 have been measured. They have not. Each is listed in the Appendix as a measurement that would settle it, with the measurement named, not waved at (6).

What is claimed is smaller, and it is checkable: a curved map with basins, a judged atom gate that admits exactly one atom per passed run, a coherence score that weights selection and nothing more, a cache placed by predicted reuse and evicted by geometry, three stores kept separate by job, and a prefix witness that proves prefix recall and nothing beyond it. If those hold, the theory is worth measuring. If they do not, the theory falls, and this paper will have said, in advance, exactly where to push (6).

Appendix: proofs and working

Appendix frozen 2026-11-04; corrections go in a new version, never by silent edit.

Each proof below follows the same shape: statement, working in numbered steps, frozen inputs, and what would settle it. Labels are part of the proof. THEORY means the claim is adopted as theory and has not been measured. PROPOSED means no code implements it yet. CODE means the identifiers and the rule below are copied from live code, and the code path is given. Nothing labelled THEORY or PROPOSED is presented as a measured result.

Proof (1): The Lorentz hyperboloid, with Whitehead spheres as basins — THEORY

Statement (theory). The mind map is the Lorentz hyperboloid. Basins on it are Whitehead spheres. Atoms live in basins. Distance on the map decides nearness, not list order.

Working.

1. Adopt Minkowski space $\mathbb{R}^{n,1}$ with Lorentz inner product $\langle x, y\rangle_L = -x_0y_0 + x_1y_1 + \cdots + x_ny_n$.
2. Define the hyperboloid $\mathcal{H} = \{x \in \mathbb{R}^{n,1} : \langle x, x\rangle_L = -1,\ x_0 > 0\}$. This is the map. Every point of the mind is a point of $\mathcal{H}$, by this definition.
3. Define Lorentz distance between two map points by $d_L(x, y) = \operatorname{arcosh}(-\langle x, y\rangle_L)$. Because $\langle x, y\rangle_L \leq -1$ on $\mathcal{H}$, the argument of arcosh is at least 1, so d_L is real and non-negative. This is the working: distance is computed, not asserted.
4. Define a basin with centre $c \in \mathcal{H}$ and radius $r_c \geq 0$ as $B(c, r_c) = \{x \in \mathcal{H} : d_L(x, c) \leq r_c\}$. In this theory, basins are called Whitehead spheres: closed balls on the hyperboloid, holding one family of atoms.
5. Membership is then a check, not a judgement call: an atom at x belongs to basin $B(c, r_c)$ if and only if $d_L(x, c) \leq r_c$. Two basins overlap exactly when some point satisfies both inequalities. Nearness is d_L, and nothing else.
6. Nesting follows from the same check: a basin $B(c_1, r_1)$ sits inside $B(c_2, r_2)$ when every x with $d_L(x, c_1) \leq r_1$ also satisfies $d_L(x, c_2) \leq r_2$. The map therefore supports hierarchy by geometry, without a separate hierarchy store.

Frozen inputs. None from code. This proof is theory adopted in this paper, not code executed by the live system. No measurement of basin membership, basin radius, or map distance on live atoms is reported here.

What would settle it. A measurement that places real atoms on $\mathcal{H}$, computes d_L between held-out related and unrelated pairs, and shows related pairs systematically closer than unrelated pairs, with basin radii chosen before the test, not after. Until that run exists, (1) is theory.

Proof (2): Toroidal phase coherence — THEORY, and a bound on what it is not

Statement (theory, with a hard bound). Atom selection is weighted by the coherence score $s = \cos(\Delta\theta_f) + \cos(\Delta\theta_s)$, where $\Delta\theta_f$ and $\Delta\theta_s$ are fast and slow phase differences between a candidate atom and the current

phase. This score is ONLY a coherence score. It is NOT a transformer attention layer, and there is NO end-to-end rotary-position-embedding replacement benchmark.

Working.

1. For a candidate atom a, let $\Delta\theta_f(a)$ and $\Delta\theta_s(a)$ be its fast and slow phase differences. Both are angles, so each cosine lies in $[-1, 1]$.
2. Compute $s(a) = \cos(\Delta\theta_f(a)) + \cos(\Delta\theta_s(a))$. By step 1, $s(a) \in [-2, 2]$. Worked arithmetic, not a measured result: if both differences are 0, $s = 1 + 1 = 2$; if both are π, $s = (-1) + (-1) = -2$; if one is 0 and the other is π, $s = 0$.
3. Weight selection by s: candidates with higher s rank above candidates with lower s, all else equal. That is the entire selection rule in this theory. No query-key product, no softmax over attention logits, and no position rotation is computed by this score, so it cannot be an attention layer by construction.
4. Bench kernels named in prior work — TPE, per-layer gain analysis, rank-5 work — are bench-crate kernels, not a transformer forward pass. They are therefore not cited here as evidence for attention replacement. Upgrading a kernel into a submission claim would require an end-to-end forward-pass benchmark, which does not exist for this score.

Frozen inputs. The formula $s = \cos(\Delta\theta_f) + \cos(\Delta\theta_s)$ as stated. No measured phase values, no measured selection gain, and no benchmark number are reported for this score in this paper.

What would settle it. A frozen suite in which atoms are selected with and without the score, on held-out tasks, with selection quality and answer quality measured separately. The score earns its place only if selection improves without answer quality being smuggled in as the same number.

Proof (3): The atom-intake gate — CODE, exactly one atom per passed run

Statement (code). The atom-intake gate is deterministic and judge-gated, admits exactly one atom per passed run, and is distinct from the prompt-adoption gate. Admission does not move confidence; only a verified outcome does.

Working.

1. A proposal arrives as a `ProposerEnvelope` with fields, in order: `family`, `strategy_id`, `policy_version`, `prefix_identity`, `map_identity`, `payload`, `expiry_ms`. Its family is one of `Retrieval`, `Prefill`, `Mtp`, `Drafter`. Code: `ares-backend-loop/src/harness/automaton.rs`.
2. State is a `StrategyState` with fields: `strategy_id`, `policy_version`, `prefix_identity`, `map_identity`, `confidence`, `confidence_floor`, `calibration_count`, `outcome_history_digest`. Confidence is `Q32_31` fixed point.
3. The transition `transition(state, proposal, signals, now_ms)` returns, in order: `Reject` if `proposal.policy_vers` differs from state, or `prefix_identity` differs, or `map_identity` differs, or `now_ms > proposal.expiry_ms`; else `Clarify` if `signals.unresolved_dependencies > 0`, or `signals.entity_ambiguity`, or `signals.measured_confi < state.confidence_floor`; else `Admit`.
4. Admission does not change confidence. The transition record carries `updated_confidence` equal to the prior confidence. A transition that admitted and also raised confidence would be self-certification, and the code refuses that shape by construction.

5. Only `update_from_outcome(state, verified, success)` moves confidence, and it returns an error and leaves state untouched when `verified` is false. When verified, with step $q = \texttt{Q32_31} :: \texttt{from_f64}(0.1)$, success sets $c \leftarrow c + (1 - c) \cdot q$ and failure sets $c \leftarrow c \cdot (1 - q)$; `calibration_count` increases by one; `outcome_history_digest` is recomputed from the prior digest, the word `success` or `failure`, and the new count, hashed with SHA-256.

6. Re-ranking is `rerank_strategies`: highest confidence first, ties broken by `strategy_id` ascending. No model, no prose, and no self-reported score enters that order.

7. Intake versus adoption: intake is the gate in steps 1–5, deciding what knowledge enters as an atom, exactly one per passed run by the gate's own rule. Prompt adoption is a different gate, deciding what guidance is followed. They share no verdict, no state field, and no acceptance rule in this proof, and must not be cited as one gate's evidence for the other.

Frozen inputs. Field names, verdict names, and the transition order above, copied from `automaton.rs` and verified against the live working copy on 2026-11-04. The envelope digest is SHA-256 over the seven envelope fields joined by newlines, in the order in step 1.

What would settle it. Nothing further is needed for the rule itself; it is code. What remains to be measured is intake in operation: how often passed runs produce exactly one atom that survives later audit, versus atoms that pass and are later withdrawn.

Proof (4): Hyperbolic nesting of the KeyValueCache — THEORY, with PROPOSED measurement

Statement (theory). Tensor KV is nested in the multi-model map. Radius equals predicted reuse. Low-reuse pages drift to the boundary and fall off. Needed pages are refilled closest to the centre. Eviction is by geometry, not deletion. Reused KV needs no eviction or compaction.

Working.

1. Place each KV page at a point of the map from Proof (1), with a radius $r \in [0, 1)$. By definition in this theory, r is the page's predicted reuse, inverted in position: high predicted reuse sits near $r = 0$ (the centre); low predicted reuse sits near $r \to 1$ (the boundary).

2. Update: as a run proceeds, recompute predicted reuse for live pages and move each page to its new radius. Movement is the update; no page is deleted by the update step.

3. Drift: a page whose predicted reuse falls moves outward, r increasing toward 1. A page whose predicted reuse rises moves inward, r decreasing toward 0. Drift is continuous in r; there is no threshold deletion step in this theory.

4. Fall-off: a page reaching the boundary, $r \to 1$, leaves the live cache. That is eviction. It is not deletion of a stored record by an operator, and it triggers no compaction of the remaining pages, because the remaining pages' positions are unchanged by a departure.

5. Refill: a needed page is placed closest to the centre among free positions, that is, at the smallest available r. A page that is being reused stays at its radius, with no eviction and no compaction applied to it.

6. Anti-lobotomy, stated as a consequence: because departure is by boundary drift and not by cutting, the surviving cache keeps its geometry. What remains is still positioned by predicted reuse, so the next retrieval still walks a map, not a compacted remainder.

Frozen inputs. None from code. No code implements radius-versus-reuse placement, drift, or boundary fall-off in the live system as of 2026-11-04. This proof is theory plus a proposed algorithm, and it is labelled as such throughout.

What would settle it (PROPOSED measurements). (a) Radius-versus-reuse tracking: log r and actual later reuse for held-out pages, and show high-r pages are in fact reused less. (b) Batch and memory at equal quality: compare nested-KV runs against a flat cache at the same answer-quality bar, and report batch size and memory, not speed alone. (c) Boundary numerics near radius 1: show placement and fall-off remain stable and precise as $r \to 1$, with the numerical error stated, not hidden. Until (a)–(c) run, (4) is theory.

Proof (5): Three stores, and the prefix-witness bound — CODE for the split, BOUND for the gate

Statement. SurrealDB holds tensor KV nested in the map. Qdrant holds retrieval with provenance. HyperDX holds telemetry, observation only. The Glimmer KV gate proves prefix recall only.

Working.

1. Store split, by job: tensor KV nested in the multi-model map lives in SurrealDB; retrieval of memories and facts with provenance lives in Qdrant; telemetry lives in HyperDX. Three stores by job, never collapsed. A claim that one store does another's job must show that job's own test passing, not merely that the store can hold the data.

2. Retrieval stays regular: Qdrant retrieval is BM25 plus cosine similarity, as it already is. No hyperbolic conversion of the retrieval store or the memory store is claimed, per the ruling of 2026-10-27. A future conversion would be a separate claim, with its own proof.

3. Telemetry is observation only: a HyperDX row records a transition or a confidence move; it does not make the transition and does not move confidence. Any argument that treats a telemetry row as a verdict confuses the record with the decision.

4. Prefix witness, by scope: the Glimmer KV gate verified prefix recall through the gate, including restart and rejection on digest change or missing heads. Scope logic then bounds the claim: a test that verifies prefix recall proves prefix recall. It does not prove distilled knowledge, because distillation would require showing knowledge that was not in the prefix. It does not prove answer quality, because quality was not the measured variable. It does not prove model-neutral decoding, because the gate was shown for one seat, not for all seats.

5. Interpretation accuracy is unmeasured. The gate is a prefix witness, and the paper reports it at exactly that strength. Any sentence that upgrades it to knowledge, quality, or neutrality is outside this proof and is not made.

Frozen inputs. The store split and the 2026-10-27 retrieval ruling, as stated. No new store measurement is reported in this paper.

What would settle it. For the gate to grow beyond prefix witness, three separate measurements: distilled-knowledge recall without the prefix present, answer quality on a held-out suite, and the same gate passing on a second, different model seat. Each is a separate claim, and none is claimed here.

Proof (6): One envelope for MTP and drafter, and what the transition does not prove — CODE

Statement (code). Multi-token prediction and drafter proposals share one envelope, one transition, and one confidence order. The transition proves admission, not quality, and not unification with any external speculative server.

Working.

1. By cases over `ProposerFamily`: the variants are exactly `Retrieval`, `Prefill`, `Mtp`, `Drafter`. Both `Mtp` and `Drafter` are variants of the same enum, carried by the same `ProposerEnvelope`, judged by the same `transition` in Proof (3), and ranked by the same `rerank_strategies` order. That shared envelope, transition, and order is the unification mechanism in code.

2. The draft policy advertised by the live native multi-token server is: execution `Mtp`, draft width 4, stream capacity 1, `max_proposers` 1, `work_limit` 3, `initial_mass` 2147483647, `admission_mass` 268435455, `reward_mass` 536870911, `qualification_trials` 64, `probe_every_epochs` 2, layer group 40. Code: draft policy fields in the native server, and the live information frame.

3. What the transition does not prove: `Admit` proves identity, expiry, dependencies, ambiguity, and floor, in that order. It does not prove that the admitted proposal is good, true, or useful. Quality is proved only by a verified outcome afterwards, through `update_from_outcome`, and even then only for that strategy, by the multiplicative step in Proof (3).

4. External bound: an external speculative drafter scheme is a separate server scheme. It is not merged into the native multi-token loop by the envelope alone, and no merger is claimed here. A merger would need its own admission, verification, and acceptance measurement, under the same transition, before it could be cited.

Frozen inputs. The enum variants, the transition rule, and the draft policy values above, verified against live code and the live server information frame.

What would settle it. A run in which `Mtp` and `Drafter` proposals compete in one held set, under one transition, with acceptance and cost reported per proposer type. Until then, unification in this paper means the shared envelope and transition, and nothing larger.

What was dropped for lack of verification

No measured number is reported in this paper that could not be verified against live code or a stated ruling. In particular: no cross-model transfer number, no answer-quality gain, no toroidal attention benchmark, no radius-versus-reuse measurement, no batch or memory comparison at equal quality, and no boundary-numerics result near radius 1 are included, because none exists to cite. Each is listed above as a future measurement instead. No external paper is cited with a page, volume, or DOI, because no external claim in this paper depends on one.

References

Live code and rulings only. No external citation with page, volume, or DOI is used.

- `ares-backend-loop/src/harness/automaton.rs` — `ProposerFamily`, `ProposerEnvelope`, `AuditSignals`, `StrategyState`, `Verdict` (`Admit` / `Reject` / `Clarify`), `transition`, `update_from_outcome`, `rerank_strategies`, `verify_mcp_authority`.
- Native multi-token server, draft policy fields and live information frame — execution `Mtp`, width 4, masses and trials in Proof (6).
- Store split and retrieval ruling, 2026-10-27 — SurrealDB for tensor KV in the map, Qdrant for retrieval with provenance, HyperDX for telemetry; regular BM25 plus cosine retrieval unchanged.
- Glimmer key-value gate, prefix witness scope — prefix recall verified; distilled knowledge, answer quality, and model-neutral decoding not claimed.